10
Behavioral Patterns

In this chapter, we explore two new design patterns from the well-known **Gang of Four** (**GoF**). They are behavioral patterns, which means that they help simplify the management of system behaviors.

Often, you need to encapsulate some core algorithm while allowing other pieces of code to extend that implementation. That is where the **Template Method** pattern comes into play.

Other times, you have a complex process with multiple algorithms that all apply to one or more situations and you need to organize it in a testable and extensible fashion. This is where the **Chain of Responsibility** pattern can help. For example, the ASP.NET Core middleware pipeline is a Chain of Responsibility where all the middleware inspects the request and acts on it.

The following topics are covered in this chapter:

- Implementing the Template Method pattern
- Implementing the Chain of Responsibility pattern
- How to mix them

Implementing the Template Method pattern

The **Template Method** is a GoF behavioral pattern that uses inheritance to share code between the base class and its subclasses. It is a very powerful, yet simple, design pattern.

Goal

The goal of the Template Method pattern is to encapsulate the outline of an algorithm in a base class while leaving some parts of that algorithm open for modification by the subclasses, which adds flexibility at a low cost.

Design

As mentioned earlier, the design is simple but extensible. First, we need to define a base class that contains the `TemplateMethod()` method and then defines one or more sub-operations that need to be implemented by its subclasses (`abstract`), or that can be overridden (`virtual`).

Using UML, it looks like this:

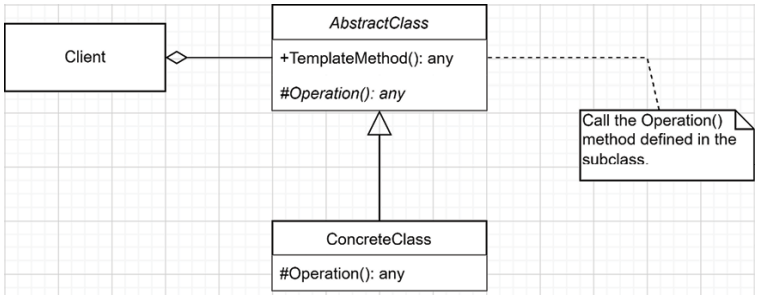

Figure 10.1: Class diagram representing the Template Method pattern

How does this work?

- `AbstractClass` implements the shared code: the algorithm.
- `ConcreteClass` implements its specific part of the algorithm.
- `Client` calls `TemplateMethod()`, which calls the subclass implementation of one or more specific algorithm elements.

> **Note**
>
> We could also extract an interface from `AbstractClass` to allow even more flexibility, but that's beyond the scope of the Template Method pattern.

Let's now get into some code to see the Template Method pattern in action.

Project – Building a search machine

Let's start with a simple, classic example to demonstrate how the Template Method pattern works.

Context: We want to use a different search algorithm depending on the collection to be searched. When the collection is sorted, we want to use a binary search, but when it is not, we want to use a linear search.

Let's start with the consumer, `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddSingleton<SearchMachine>(x => new LinearSearchMachine(1, 10, 5, 2, 123, 333, 
    .AddSingleton<SearchMachine>(x => new BinarySearchMachine(1, 2, 3, 4, 5, 6, 7, 8,
;
var app = builder.Build();
app.MapGet("/", (IEnumerable<SearchMachine> searchMachines) =>
{
    var sb = new StringBuilder();
    var elementsToFind = new int[] { 1, 10, 11 };
    foreach (var searchMachine in searchMachines)
    {
        var heading = $"Current search machine is {searchMachine.GetType().Name}";
        sb.AppendLine("".PadRight(heading.Length, '='));
        sb.AppendLine(heading);
        foreach (var value in elementsToFind)
        {
            var index = searchMachine.IndexOf(value);
            var wasFound = index.HasValue;
            if (wasFound)
```

```
            {
                sb.AppendLine($"The element '{value}' was found at index {index!.Valu
            }
            else
            {
                sb.AppendLine($"The element '{value}' was not found.");
            }
        }
    }
    return sb.ToString();
});
app.Run();
```

In the consumer code, we configure `LinearSearchMachine` and `BinarySearchMachine` as two `SearchMachine` implementations. Each instance is initialized using a different sequence of numbers.

We then inject all registered `SearchMachine` services into the `/` delegate (highlighted in the code block). That handler iterates all `SearchMachine` instances and tries to find all elements of the `elementsToFind` array, before outputting the `text/plain` results.

Next, let's explore the `SearchMachine` class:

```
namespace TemplateMethod;
public abstract class SearchMachine
{
    protected int[] Values { get; }
    protected SearchMachine(params int[] values)
    {
        Values = values ?? throw new ArgumentNullException(nameof(values));
    }
    public int? IndexOf(int value)
    {
        if (Values.Length == 0) { return null; }
        var result = Find(value);
        return result;
    }
    protected abstract int? Find(int value);
}
```

The `SearchMachine` class represents `AbstractClass`. It exposes the `IndexOf()` template method, which uses the required hook represented by the `abstract Find()` method (see highlighted code). The hook is required because each subclass must implement that method, thereby making that method a required extension point (or hook).

Next, we explore our first implementation of `ConcreteClass`, the `LinearSearchMachine` class:

```
namespace TemplateMethod;
public class LinearSearchMachine : SearchMachine
{
    public LinearSearchMachine(params int[] values)
        : base(values) { }
    protected override int? Find(int value)
    {
        var index = 0;
        foreach (var item in Values)
        {
            if (item == value) { return index; }
```

```
            index++;
        }
        return null;
    }
}
```

The `LinearSearchMachine` class is a `ConcreteClass` representing the linear search implementation used by `SearchMachine`. It's part of the algorithm implemented by the `Find` method.

Finally, we move on to the `BinarySearchMachine` class:

```
namespace TemplateMethod;
public class BinarySearchMachine : SearchMachine
{
    public BinarySearchMachine(params int[] values)
        : base(values.OrderBy(v => v).ToArray()) { }
    protected override int? Find(int value)
    {
        var index = Array.BinarySearch(Values, value);
        return index == -1 ? null : index;
    }
}
```

The `BinarySearchMachine` class is a `ConcreteClass` representing the binary search implementation of `SearchMachine`. As you may have noticed, we skipped the binary search algorithm's implementation by delegating it to the built-in `Array.BinarySearch` method. Thanks to the .NET team!

> **Important**
>
> For a binary search algorithm to work, the collection must be sorted, hence the sorting that is done in the constructor when passing the values to the base class (`OrderBy`). That may not be the most performant way of ensuring the array is sorted (precondition/guard), but it is a very fast and readable way to write it. Moreover, in our case, performance should not be an issue. Furthermore, to optimize such an algorithm, you need to test a large set of data and leverage parallelism (multithreading).

Now that we have defined the actors and explored the code, let's see what is happening in `client`:

1. `client` uses the registered `SearchMachine` instances and searches for a set of values (1, 10, and 11).
2. Once that is done, `client` displays to the user whether the numbers were found or not.

In this case, the template method and the `Find` method return `null` when a value is not found.

By running the program, we get the following output:

```
================================================
Current search machine is LinearSearchMachine
The element '1' was found at index 0.
```

```
The element '10' was found at index 1.
The element '11' was not found.
==========================================
Current search machine is BinarySearchMachine
The element '1' was found at index 0.
The element '10' was found at index 9.
The element '11' was not found.
```

The preceding output shows the two algorithms at play. Both `SearchMachine` implementations did not contain the value `11`. They both contained the values `1` and `10` placed at a different position of their respective arrays. Here is a reminder of the values:

```
new LinearSearchMachine(1, 10, 5, 2, 123, 333, 4)
new BinarySearchMachine(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

The consumer was iterating the `SearchMachine` registered with the IoC container. The base class implements the `IndexOf` but delegates the search (`Find`) algorithm to the subclasses. The preceding output shows that each `SearchMachine` was able to execute the expected task by implementing only the `Find` piece of the algorithm.

And voilà! We have covered the Template Method pattern, as easy as that. Of course, our algorithm was very simple, but the concept remains.

We could add `virtual` methods in the base class to create optional hooks. Those methods would become optional extension points that subclasses can implement or not. That would allow a more complex and more versatile scenario to be supported. We will not cover this here because it is not part of the pattern itself, even if very similar. There are many examples in the .NET **base class library** (**BCL**), like most methods of the `ComponentBase` class (in the `Microsoft.AspNetCore.Components` namespace). For example, when we override the `OnInitialized` method in a Blazor component, we leverage such an optional extension hook. The base method does nothing and is there only for extensibility purposes, allowing us to run code as part of the component's lifecycle. You can consult the `ComponentBase` class code in the official .NET repo on GitHub: https://adpg.link/1WYq.

Conclusion

The Template Method is a powerful and easy-to-implement design pattern that allows subclasses to reuse the algorithm's skeleton while implementing (`abstract`) or overriding (`virtual`) some of its subparts. Allowing implementation-specific classes to extend the core algorithm can reduce the duplication of logic and improve maintainability while not cutting out any flexibility in the process. There are many examples in the .NET BCL, and we leverage this pattern at the end of the chapter, based on a real-world scenario.

Now, let's see how the Template Method pattern can help us follow the **SOLID** principles:

- **S**: The Template Method pushes algorithm-specific portions of the code to subclasses while keeping the core algorithm in the base class. By do-

ing that, it helps follow the **single responsibility principle** (**SRP**) by distributing responsibilities.

- **O**: By opening extension hooks, it opens the template for extensions (allowing subclasses to extend it) and closes it from modifications (no need to modify the base class since the subclasses can extend it).
- **L**: Since the subclasses are the implementations, there are no base behaviors to enforce in subclasses, so following the **Liskov substitution principle** (**LSP**) should not be a problem when implementing the Template Method pattern. That said, this principle is tricky, so it could be possible to create a subclass (or a subclass of a subclass) that breaks the LSP, thereby altering the program logic.
- **I**: As long as the base class implements the smallest cohesive surface possible, using the Template Method pattern should not negatively impact the program.
- **D**: The Template Method pattern is based on an abstraction, so as long as consumers depend on that abstraction, it should help to get in line with the **dependency inversion principle** (**DIP**).

Next, we move to the Chain of Responsibility design pattern before mixing the Template Method and the Chain of Responsibility pattern to improve our code.

Implementing the Chain of Responsibility pattern

The **Chain of Responsibility** is a GoF behavioral pattern that chains classes to handle complex scenarios efficiently, with limited effort. Once again, the goal is to take a complex problem and break it into multiple smaller units.

Goal

The goal of the Chain of Responsibility pattern is to chain multiple handlers that each solve a limited number of problems. If a handler cannot solve the specific problem, it passes the resolution to the chain's next handler. There could be a default handler executing some logic at the end of the chain, such as throwing a custom exception (for example, `OperationNotHandledException`) or a handler that makes sure of the opposite (in other words, that nothing happens, especially no exception).

Design

The most basic Chain of Responsibility starts by defining an interface that handles a request (`IHandler`). Then we add classes that handle one or more scenarios (`Handler1` and `Handler2`):

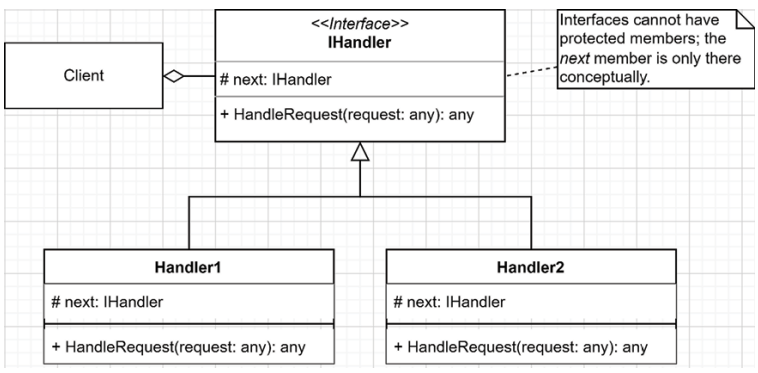

Figure 10.2: Class diagram representing the Chain of Responsibility pattern

> **Tip**
>
> When creating the chain of responsibility, you can order the handlers so that the most requested handlers are closer to the beginning of the chain and the least requested handlers are closer to the end. This helps limit the number of "chain links" that are visited for each request before reaching the right handler.

A big difference between the Chain of Responsibility pattern and many other patterns is that no central dispatcher knows the handlers; all handlers are independent. The consumer receives a handler and tells it to handle the request, no more complexity than this. Each handler is also simple, handling the request or not, and then passing it, or not, to the next handler in the chain. This allows us to divide complex logic into multiple pieces that handle a single responsibility, improving testability, reusability, and extensibility in the process. Since there is no orchestrator, each chain element is independent, leading to a cohesive and loosely coupled design.

Enough theory. Let's look at some code.

Project – Message interpreter

Context: We need to create the receiving end of a messaging application where each message is unique, making it impossible to create a single algorithm to handle them all.

After analyzing the problem, we decided to build a chain of responsibility where each handler can manage a single message. The pattern seems more than perfect!

> **Background**
>
> This project is based on something that I built years ago. Physical (IoT) devices were sending bytes (messages) due to limited bandwidth. Then, in a web application, we had to associate those bytes with real values. Each message had a fixed header size, but a variable body size. The headers were handled in a base handler (template method), and each individual handler in the chain was managing a different message type. For the current example, we keep it simpler than parsing bytes, but the concept is the same.

For our demo application, the messages are as simple as this:

```
namespace ChainOfResponsibility;
public record class Message(string Name, string? Payload);
```

The `Name` property is used as a discriminator to differentiate messages, and each handler's responsibility is to do something with the `Payload` property. We won't do anything with the payload as it is irrelevant to the pattern, but conceptually, that's what should happen.

The handlers are also very simple:

```
namespace ChainOfResponsibility;
public interface IMessageHandler
{
    void Handle(Message message);
}
```

The only thing a handler can do is handle a message. Our initial application can handle the following messages:

- The `AlarmTriggeredHandler` class handles `AlarmTriggered` messages.
- The `AlarmPausedHandler` class handles `AlarmPaused` messages.
- The `AlarmStoppedHandler` class handles `AlarmStopped` messages.

The three handlers are very similar and share quite a bit of logic, but we fix that later. In the meantime, we have the following:

```
public class AlarmTriggeredHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmTriggeredHandler(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (message.Name == "AlarmTriggered")
        {
            // Do something clever with the Payload
        }
        else if (_next != null)
        {
            _next.Handle(message);
        }
    }
}
public class AlarmPausedHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmPausedHandler(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (message.Name == "AlarmPaused")
        {
            // Do something clever with the Payload
        }
        else if (_next != null)
        {
            _next.Handle(message);
        }
    }
}
public class AlarmStoppedHandler : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public AlarmStoppedHandler(IMessageHandler? next = null)
    {
```

```
            _next = next;
        }
        public void Handle(Message message)
        {
            if (message.Name == "AlarmStopped")
            {
                // Do something clever with the Payload
            }
            else if (_next != null)
            {
                _next.Handle(message);
            }
        }
    }
```

Each handler does two things:

- It allows an optional "next handler" to be injected into its constructor (highlighted in the code). The handler classes use that `_next` member in the `Handle` method.
- It handles only the request that it knows about, delegating the others to the next handler in the chain (the `Handle` method).

In this case, if the next handler is null, nothing happens. In a real scenario, you may want to know that a handler is missing or that the message was invalid. Let's add a fourth handler that notifies us of invalid requests:

```
public class DefaultHandler : IMessageHandler
{
    public void Handle(Message message)
    {
        throw new NotSupportedException($"Messages named '{message.Name}' are not supp
    }
}
```

That new default handler throws an exception that notifies the consumer of the Chain of Responsibility about the error.

Note

We can create custom exceptions to make it easier to differentiate between system errors and application errors. But sometimes, throwing a system exception is good enough. For example, here are a few exceptions that are often thrown as is: `NotSupportedException`, `NotImplementedException`, and `ArgumentNullException`.

Let's use `Program.cs` as the consumer of the Chain of Responsibility (the client) and use HTTP requests as the way to interface with our program and build the message.

Tip

Usually, the `GET` method is used to read data, while other methods, such as `POST`, `PUT`, and `PATCH`, are used to create, replace, or update data. For testing purposes, it is easier

> to send arbitrary data to our Chain of Responsibility by using `GET` (I recommend that you don't do that in your apps), so we are cheating on this one.

Here is our program, using minimal APIs and top-level statements:

```
var builder = WebApplication.CreateBuilder(args);
// Create the chain of responsibility,
// ordered by the most called (or the ones to execute earlier)
// to the less called handler (or the ones that take more time to execute),
// followed by the DefaultHandler.
builder.Services.AddSingleton<IMessageHandler>(new AlarmTriggeredHandler(new AlarmPaus
```

In the preceding code, we manually create the Chain of Responsibility and register it as a singleton of `IMessageHandler`. In that registration code, each handler is injected in the previous constructor manually (created with the `new` keyword).

The next code represents a list of relative URLs that are displayed when calling `/`, for convenience purposes:

```
var app = builder.Build();
// "Menu" endpoint
app.MapGet("/", () => new[] {
    "/handle/AlarmTriggered",
    "/handle/AlarmPaused",
    "/handle/AlarmStopped",
    "/handle/SomeUnhandledMessageName",
});
```

The following code is the consumer of the chain:

```
// Consumer (client) endpoint
app.MapGet("/handle/{name}", (string name, string? payload, IMessageHandler messageHa
{
    var message = new Message(name, payload);
    try
    {
        // Send the message into the chain of responsibility
        messageHandler.Handle(message);
        return $"Message '{message.Name}' handled successfully.";
    }
    catch (NotSupportedException ex)
    {
        return ex.Message;
    }
});
app.Run();
```

The consumer is reachable through the `/handle/{name}` URL. The delegate expects the `name`, an optional `payload`, and an `IMessageHandler` implementation to be injected. For every request, it does the following:

1. Creates a `Message` based on the `name` and `payload` parameters.
2. Passes that message to the first handler of the chain of responsibility (injected into the `/handle/{name}` delegate): `messageHandler.Handle(message);`.

3. Writes an error message when a `NotSupportedException` has been thrown, or a success message otherwise.

When running the application, we should expect the following "menu" when calling `https://localhost:10001/`:

Figure 10.3: The root endpoint JSON response that represents a menu

By specifying a valid `name` such as `AlarmTriggered`, we should obtain the following result:

```
URL: https://localhost:10001/AlarmTriggered
Message 'AlarmTriggered' handled successfully.
```

By specifying an invalid `name` such as `SomeUnhandledMessageName`, we should obtain the following result:

```
URL: https://localhost:10001/SomeUnhandledMessageName
Messages named 'SomeUnhandledMessageName' are not supported.
```

And voilà. We have built a simple Chain of Responsibility that handles messages. Next, let's use both the Template Method and Chain of Responsibility patterns to encapsulate our handlers' duplicated logic.

Project – Improved message interpreter

Now that we know both the **Chain of Responsibility** and the **Template Method** patterns, it is time to *DRY* out our handlers by extracting the shared logic into an abstract base class using the Template Method pattern and providing extension points to the subclasses.

DRY

We covered the **Don't Repeat Yourself** (**DRY**) principle in *Chapter 3, Architectural Principles*.

OK, so what has been duplicated?

- The `next` handler injection code has been duplicated, and, as an important part of the pattern, could be encapsulated into the base class.
- The logic testing whether the current handler can handle the message has been duplicated.

The new base class looks like this:

```
namespace ImprovedChainOfResponsibility;
public abstract class MessageHandlerBase : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public MessageHandlerBase(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (CanHandle(message))
        {
            Process(message);
        }
        else if (HasNext())
        {
            _next.Handle(message);
        }
    }
    [MemberNotNullWhen(true, nameof(_next))]
    private bool HasNext()
    {
        return _next != null;
    }
    protected virtual bool CanHandle(Message message)
    {
        return message.Name == HandledMessageName;
    }
    protected abstract string HandledMessageName { get; }
    protected abstract void Process(Message message);
}
```

Based on those few changes, what is the template method, and what are the extension points (hooks)?

The `MessageHandlerBase` class adds the `Handle` template method. That template method's algorithm is easier to read than it was before. Then, `MessageHandlerBase` exposes the following extension points:

- `CanHandleMessage(Message message)` tests whether `HandledMessageName` is equal to `message.Name`. This could be overridden if a handler required more complex comparison logic.
- `HandledMessageName` must be implemented by all subclasses, driving the default logic of `CanHandleMessage`.
- `Process(Message message)` must be implemented by all subclasses, allowing them to run logic against the message.

Let's now take a look at the three simplified alarm handlers:

```
public class AlarmTriggeredHandler : MessageHandlerBase
{
    protected override string HandledMessageName => "AlarmTriggered";
    public AlarmTriggeredHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmPausedHandler : MessageHandlerBase
{
```

```
    protected override string HandledMessageName => "AlarmPaused";
    public AlarmPausedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
public class AlarmStoppedHandler : MessageHandlerBase
{
    protected override string HandledMessageName => "AlarmStopped";
    public AlarmStoppedHandler(IMessageHandler? next = null)
        : base(next) { }
    protected override void Process(Message message)
    {
        // Do something clever with the Payload
    }
}
```

As we can see from the updated alarm handlers, they are now limited to a single responsibility: processing the messages they can handle. In contrast, `MessageHandlerBase` now handles the chain of responsibility's plumbing. We left the `DefaultHandler` class unchanged since it is the end of the chain and does not support having a next handler.

By mixing those two patterns, we created a complex messaging system that divides responsibilities into handlers. There is one handler per message, and the chain logic is pushed into a base class.

The beauty of such a system is that we don't have to think about all the messages at once; we can focus on just one message at a time. When dealing with a new type of message, we can concentrate on that precise message, implement its handler, and forget about the *N* other types. The consumers can also be super dumb, sending the request into the pipe without knowing about the Chain of Responsibility, and like magic, the right handler shall prevail!

Project – A final, finer-grained design

In the last example, we were using `HandledMessageName` and `CanHandleMessage` to decide whether a handler could handle a request. There is one problem with that code: if a subclass decides to override `CanHandleMessage`, and then decides that it no longer requires `HandledMessageName`, we would end up having a lingering, unused property in our system.

Note

There are worse situations, but we are talking component design here, so why not push that system a little further toward a better design?

One way to fix this is to create a finer-grained class hierarchy, as follows:

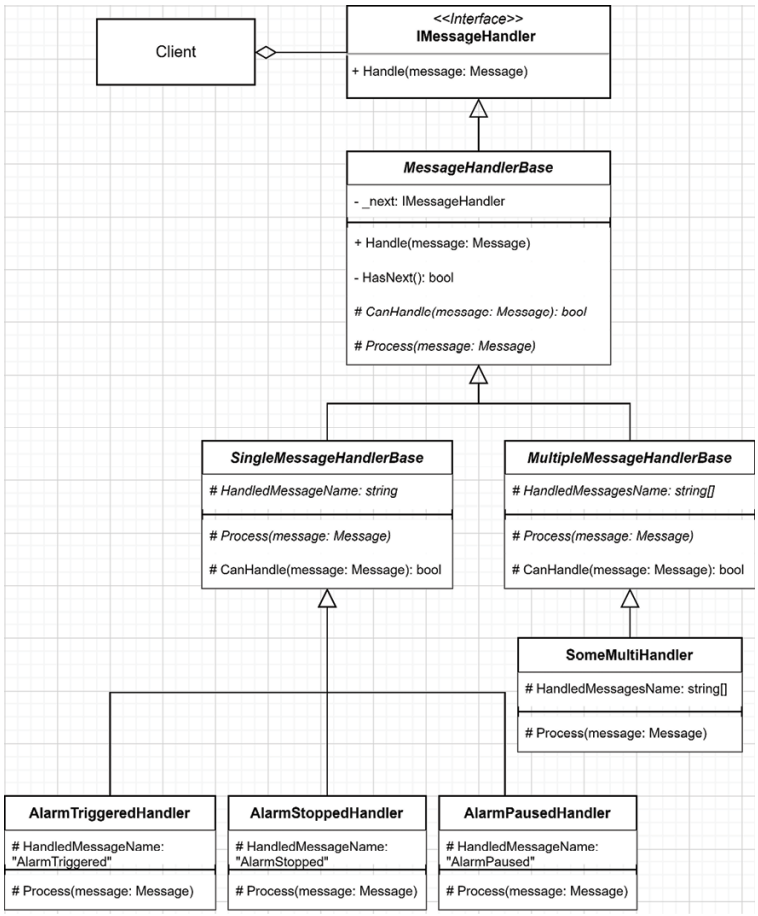

Figure 10.4: Class diagram representing the design of the finer-grained project that implements the Chain of Responsibility and Template Method patterns

That looks more complicated than it is, really. Before digging into what it actually does, let's take a look at the refactored code:

```
namespace FinalChainOfResponsibility;
public interface IMessageHandler
{
    void Handle(Message message);
}
public abstract class MessageHandlerBase : IMessageHandler
{
    private readonly IMessageHandler? _next;
    public MessageHandlerBase(IMessageHandler? next = null)
    {
        _next = next;
    }
    public void Handle(Message message)
    {
        if (CanHandle(message))
        {
            Process(message);
        }
        else if (HasNext())
        {
            _next.Handle(message);
        }
    }
    [MemberNotNullWhen(true, nameof(_next))]
    private bool HasNext()
    {
```

```
            return _next != null;
        }
        protected abstract bool CanHandle(Message message);
        protected abstract void Process(Message message);
    }
    public abstract class SingleMessageHandlerBase : MessageHandlerBase
    {
        public SingleMessageHandlerBase(IMessageHandler? next = null)
            : base(next) { }
        protected override bool CanHandle(Message message)
        {
            return message.Name == HandledMessageName;
        }
        protected abstract string HandledMessageName { get; }
    }
    public abstract class MultipleMessageHandlerBase : MessageHandlerBase
    {
        public MultipleMessageHandlerBase(IMessageHandler? next = null)
            : base(next) { }
        protected override bool CanHandle(Message message)
        {
            return HandledMessagesName.Contains(message.Name);
        }
        protected abstract string[] HandledMessagesName { get; }
    }
```

The omitted `AlarmPausedHandler`, `AlarmStoppedHandler`, and `AlarmTriggeredHandler` classes now inherit from `SingleMessageHandlerBase` instead of `MessageHandlerBase`, but nothing else has changed. `DefaultHandler` has not changed either. For demonstration purposes, I added the `SomeMultiHandler` class to simulate a message handler that can handle `"Foo"`, `"Bar"`, and `"Baz"` messages. That looks like the following:

```
namespace FinalChainOfResponsibility
{
    public class SomeMultiHandler : MultipleMessageHandlerBase
    {
        public SomeMultiHandler(IMessageHandler? next = null)
            : base(next) { }
        protected override string[] HandledMessagesName
            => new[] { "Foo", "Bar", "Baz" };
        protected override void Process(Message message)
        {
            // Do something clever with the Payload
        }
    }
}
```

Now that we have seen the code and the UML representation of the class hierarchy, let's analyze the actors of the new structure:

- `MessageHandlerBase` manages the Chain of Responsibility by handling the next handler logic and by exposing two hooks (the Template Method pattern) for subclasses to extend:
  1. `bool CanHandle(Message message)`
  2. `void Process(Message message)`
- `SingleMessageHandlerBase` inherits from `MessageHandlerBase` and implements (`override`) the `bool CanHandle(Message message)` method. It implements the logic related to it and adds the `abstract string HandledMessageName { get; }` property that

subclasses must define (`override`) for the `CanHandle` method to work (a required extension point).
- The subclasses of `SingleMessageHandlerBase` implement the `HandledMessageName` property, which returns the message name that they can handle and implements the handling logic by overriding the `void Process(Message message)` method.
- `MultipleMessageHandlerBase` does the same as `SingleMessageHandlerBase` , but it uses a string array instead of a single string, supporting multiple handler names.

This may sound complicated, but what we did was to allow extensibility without the need to keep any unnecessary code in the process, leaving each class with a single responsibility:

- `MessageHandlerBase` handles `_next` .
- `SingleMessageHandlerBase` handles the `CanHandle` method of handlers, supporting just a single message.
- `MultipleMessageHandlerBase` handles the `CanHandle` method of handlers supporting multiple messages.
- Other classes must implement their version of `void Process(Message message)` to handle one or more messages.

And voilà! This is another example demonstrating the strength of the Template Method and Chain of Responsibility patterns put together. That last example also emphasizes the importance of the SRP by allowing greater flexibility while keeping the code reliable and maintainable.

Another strength of that design is the interface at the top. Anything that does not fit the class hierarchy can be implemented directly from the interface instead of trying to adapt logic from inappropriate structures—tricking code into doing your bidding often leads to half-baked solutions that become hard to maintain. The `DefaultHandler` class is a good example of that.

Conclusion

The Chain of Responsibility pattern is another great GoF pattern. It divides a large problem into smaller, cohesive units, each doing one job: handling its specific request(s). Mixed with the Template Method pattern, it can become even simpler to handle the chain, moving each part closer toward single responsibilities.

Now, let's see how the Chain of Responsibility pattern can help us follow the **SOLID** principles:

- **S**: The Chain of Responsibility pattern aims toward this exact principle, making it a perfect SRP advocate: single units of logic per class!
- **O**: The Chain of Responsibility pattern allows the addition, removal, and reordering of handlers without touching the code, but by altering the composition of the chain (in the composition root).
- **L**: N/A
- **I**: By creating a small interface with multiple handlers (implementations), the Chain of Responsibility pattern should help with the ISP. The handler interface is not limited to a single method; it can expose multiple methods as long as they aim toward the same responsibility. Cohesion is key.

- **D**: By using the handler interface, no element of the chain, nor the consumers, depends on a specific handler; they only depend on the interface, which helps with the DIP.

Summary

In this chapter, we covered two GoF behavioral patterns. These patterns can help us create a flexible, yet easy-to-maintain system. As the name suggests, behavioral patterns aim at encapsulating application behaviors into cohesive software pieces.

First, we looked at the Template Method pattern, which allows us to encapsulate an algorithm's core inside a base class. It then allows its subclasses to fill in the gaps and extend that algorithm at predefined locations. These locations can be required (`abstract`) or optional (`virtual`).

Then, we explored the Chain of Responsibility pattern, which opens the possibility of chaining multiple small handlers into a chain of processing, inputting the message to be processed at the beginning of the chain, and waiting for one or more handlers to execute the actual logic related to that message against it. That is an important nuance: you don't have to stop the chain's execution at the first handler. In some cases, the Chain of Responsibility could become more like a pipeline than a clear association of one message to one handler.

Lastly, using the Template Method pattern to encapsulate the Chain of Responsibility's chaining logic led us to a simpler implementation without any sacrifices.

In the next chapter, we are going to dig into the Operation Result design pattern to discover efficient ways of managing return values.

Questions

Let's take a look at a few practice questions:

1. Is it true that we can only add one `abstract` method when implementing the Template Method design pattern?
2. Can we use the Strategy pattern in conjunction with the Template Method pattern?
3. Is it true that there is a limit of 32 handlers in a Chain of Responsibility?
4. In a Chain of Responsibility, can multiple handlers process the same message?
5. In what way can the Template Method help implement the Chain of Responsibility pattern?